

Understanding C# Delegates

What is a Delegate?

- A delegate in C# is a type that represents references to methods with a specific parameter list and return type. It allows methods to be passed as parameters, enabling event-driven programming and callback methods.
- Delegates are similar to function pointers in C and C++, but they are type-safe and object-oriented.

Types of Delegates

1. **Singlecast Delegates:** Point to a single method.
2. **Multicast Delegates:** Can point to multiple methods. When invoked, they call all the methods in their invocation list.

How to Define and Use Delegates

Step 1: Declare a Delegate

```
csharp
VerifyOpen In EditorRunCopy code
1public delegate int MyDelegate(int x, int y);
```

- This defines a delegate named `MyDelegate` that can reference any method that takes two integers and returns an integer.

Step 2: Instantiate a Delegate

```
csharp
VerifyOpen In EditorRunCopy code
1MyDelegate del = new MyDelegate(CalculateSum);
```

- Here, `CalculateSum` is a method that matches the delegate's signature.

Step 3: Invoke the Delegate

```
csharp
VerifyOpen In EditorRunCopy code
1int result = del(5, 10);
2Console.WriteLine(result); // Outputs: 15
```

Example of a Singlecast Delegate

Here's a complete example demonstrating a singlecast delegate:

```
csharp
VerifyOpen In EditorRunCopy code
1using System;
2
3class Program
4{
5    static int CalculateSum(int x, int y)
6    {
7        return x + y;
8    }
9
10    public delegate int MyDelegate(int num1, int num2);
```

```

11
12     static void Main()
13     {
14         MyDelegate del = new MyDelegate(CalculateSum);
15         int result = del(5, 10);
16         Console.WriteLine(result); // Outputs: 15
17     }
18 }

```

Example of a Multicast Delegate

Here's how to use a multicast delegate:

```

csharp
VerifyOpen In EditorRunCopy code
1using System;
2
3class Program
4{
5    public delegate void MyDelegate(int num1, int num2);
6
7    static void Main()
8    {
9        MyDelegate del = Sum;
10        del += Difference; // Add another method to the delegate
11        del(10, 5);
12    }
13
14    static void Sum(int x, int y)
15    {
16        Console.WriteLine("Sum: " + (x + y));
17    }
18
19    static void Difference(int x, int y)
20    {
21        Console.WriteLine("Difference: " + (x - y));
22    }
23}

```

Output:

```

VerifyOpen In EditorRunCopy code
1Sum: 15
2Difference: 5

```

Key Points About Delegates

- **Type Safety:** Delegates ensure that the method signature matches the delegate type.
- **Flexibility:** You can change the method that a delegate points to at runtime.
- **Event Handling:** Delegates are commonly used in event handling, allowing methods to be called in response to events.

Uses of Delegates in C#

1. Method Invocation

Delegates can be used to invoke methods dynamically.

Example:

```

csharp
VerifyOpen In EditorRunCopy code
1using System;
2
3public delegate void MyDelegate(string message);
4
5class Program
6{
7    public static void DisplayMessage(string message)
8    {
9        Console.WriteLine(message);
10    }
11
12    static void Main()
13    {
14        MyDelegate del = new MyDelegate(DisplayMessage);
15        del("Hello, Delegates!");
16    }
17}

```

2. Event Handling

Delegates are commonly used in event handling.

Example:

```

csharp
VerifyOpen In EditorRunCopy code
1using System;
2
3public delegate void Notify(); // Delegate
4
5public class Process
6{
7    public event Notify ProcessCompleted; // Event
8
9    public void StartProcess()
10    {
11        Console.WriteLine("Process Started!");
12        System.Threading.Thread.Sleep(2000);
13        OnProcessCompleted();
14    }
15
16    protected virtual void OnProcessCompleted()
17    {
18        ProcessCompleted?.Invoke(); // Invoke the event
19    }
20}
21
22class Program
23{
24    static void Main()
25    {
26        Process process = new Process();
27        process.ProcessCompleted += () => Console.WriteLine("Process
Completed!");
28        process.StartProcess();
29    }
30}

```

3. Multicast Delegates

Multicast delegates allow you to call multiple methods in a single delegate invocation.

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1using System;
2
3public delegate void MultiDelegate();
4
5class Program
6{
7    public static void Method1()
8    {
9        Console.WriteLine("Method 1 Executed");
10    }
11
12    public static void Method2()
13    {
14        Console.WriteLine("Method 2 Executed");
15    }
16
17    static void Main()
18    {
19        MultiDelegate del = Method1;
20        del += Method2; // Add Method2 to the delegate
21        del(); // Invoke both methods
22    }
23}
```

4. Callback Methods

Delegates can be used to implement callback methods.

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1using System;
2
3public delegate int Operation(int x, int y);
4
5class Calculator
6{
7    public int ExecuteOperation(int a, int b, Operation op)
8    {
9        return op(a, b); // Call the delegate
10    }
11}
12
13class Program
14{
15    static int Add(int x, int y) => x + y;
16    static int Subtract(int x, int y) => x - y;
17
18    static void Main()
19    {
20        Calculator calc = new Calculator();
21        Console.WriteLine("Addition: " + calc.ExecuteOperation(5, 3, Add));
22        Console.WriteLine("Subtraction: " + calc.ExecuteOperation(5, 3,
23        Subtract));
24    }
25}
```

```
23     }
24 }
```

5. Asynchronous Programming

Delegates are also useful in asynchronous programming.

Example:

```
csharp
VerifyOpen In EditorRunCopy code
1using System;
2using System.Threading;
3
4public delegate void AsyncDelegate();
5
6class Program
7{
8    public static void LongRunningOperation()
9    {
10        Console.WriteLine("Operation Started...");
11        Thread.Sleep(3000); // Simulate long operation
12        Console.WriteLine("Operation Completed!");
13    }
14
15    static void Main()
16    {
17        AsyncDelegate del = LongRunningOperation;
18        del.BeginInvoke(null, null); // Start the operation asynchronously
19        Console.WriteLine("Main thread continues...");
20        Console.ReadLine(); // Wait for user input to keep the console open
21    }
22}
```

Action, Func, and Predicate Delegates

1. Action Delegate

- **Definition:** An `Action` delegate encapsulates a method that does not return a value.
- **Example:**

```
csharp
VerifyOpen In EditorRunCopy code
1Action<int, int> printSum = (a, b) => Console.WriteLine($"Sum: {a + b}");
2printSum(5, 10); // Output: Sum: 15
```

2. Func Delegate

- **Definition:** A `Func` delegate encapsulates a method that returns a value.
- **Example:**

```
csharp
VerifyOpen In EditorRunCopy code
1Func<int, int, int> multiply = (a, b) => a * b;
2int result = multiply(5, 10);
3Console.WriteLine($"Product: {result}"); // Output: Product: 50
```

3. Predicate Delegate

- **Definition:** A `Predicate` delegate takes one input parameter and returns a boolean value.

- **Example:**

```
csharp
VerifyOpen In EditorRunCopy code
1Predicate<int> isEven = number => number % 2 == 0;
2bool result = isEven(10);
3Console.WriteLine($"Is 10 even? {result}"); // Output: Is 10 even? True
```

Conclusion

Delegates in C# provide a flexible way to encapsulate method references, enabling dynamic method invocation, event handling, and asynchronous programming. Understanding how to define, instantiate, and invoke delegates is crucial for effective C# programming.